

# Technical Documentation: Applying RAGAS Evaluation to the Biblical Counseling Chatbot

## Table of Contents

- 1. [RAGAS Origin](#)
- 2. [What RAGAS Is Missing for This Project](#)
- 3. [Step-by-Step: Modifying Vanilla RAGAS for This Project](#)
- 4. [How to Run RAGAS Evaluation in This Project](#)
- 5. [How to Read the Results](#)

## 1. RAGAS Origin

### 1.1 What Is RAGAS?

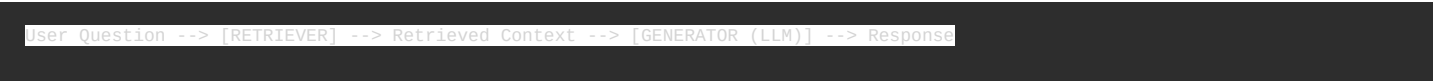
**RAGAS** (Retrieval-Augmented Generation Assessment) is an open-source evaluation framework designed to assess the quality of Retrieval-Augmented Generation (RAG) pipelines. It was developed by the team at [Explodinggradients \(now VibrantLabs AI\)](#) and published as a Python package on PyPI.

The framework was introduced in the research paper:

*Es, S., James, J., Espinosa Anke, L., & Schockaert, S. (2024). RAGAS: Automated Evaluation of Retrieval Augmented Generation. Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics (EACL 2024).*

### 1.2 The Core Idea

A standard RAG pipeline has two components that can fail independently:



RAGAS evaluates both components using four metrics, each scored from **0.0 to 1.0**:

| Metric            | What It Evaluates                                                                                            | Component |
|-------------------|--------------------------------------------------------------------------------------------------------------|-----------|
| Faithfulness      | Is the generated response factually grounded in the retrieved context? Does it avoid hallucination?          | Generator |
| Answer Relevancy  | Does the generated response directly address what the user asked?                                            | Generator |
| Context Precision | Are the retrieved documents actually relevant to the user's query? Are the most relevant ones ranked higher? | Retriever |
| Context Recall    | Does the retrieved context contain enough information to produce the correct answer?                         | Retriever |

### 1.3 How RAGAS Calculates Metrics (LLM-as-a-Judge)

RAGAS does **not** use traditional NLP metrics like BLEU or ROUGE. Instead, it uses an **LLM-as-a-Judge** approach. A separate LLM (the "judge") reads the input, context, response, and reference answer, then provides a structured assessment.

For example, to compute **Faithfulness**: 1. The judge LLM extracts all individual claims/statements from the generated response. 2. For each claim, the judge checks: "Is this claim supported by the retrieved context?" 3. Faithfulness = (number of supported claims) / (total number of claims).

This approach allows RAGAS to evaluate **semantic** quality rather than surface-level text overlap.

### 1.4 Vanilla RAGAS Usage (From the Official GitHub)

The standard usage from the RAGAS GitHub repository is straightforward:

```
from ragas import evaluate, EvaluationDataset, SingleTurnSample from ragas.metrics.collections import Faithfulness, AnswerRelevancy # Step 1: Prepare data (you must already have all four fields) sample = SingleTurnSample( user_input="What is the capital of France?", retrieved_contexts=["Paris is the capital of France."], response="The capital of France is Paris.", reference="Paris" ) dataset = EvaluationDataset(samples=[sample]) # Step 2: Evaluate (defaults to OpenAI GPT-4 as judge) results = evaluate(dataset=dataset, metrics=[Faithfulness(), AnswerRelevancy()]) # Step 3: View results print(results.to_pandas())
```

This vanilla approach assumes: - You already have a complete dataset with all four fields pre-filled. - Your RAG system is a simple "retrieve from document store, then answer" pipeline. - You have an OpenAI API key for the default GPT-4 judge.

## 2. What RAGAS Is Missing for This Project

The Biblical Counseling Chatbot is **not** a standard RAG system. It has a multi-stage counseling flow, dual retrieval sources, and domain-specific requirements. Below are the five gaps between vanilla RAGAS and this project's needs.

### 2.1 Gap: No Pre-Built Ground Truth Dataset

**The Problem:** Vanilla RAGAS expects a ready-made CSV with four columns: `user_input`, `retrieved_contexts`, `response`, and `reference`. This project has no such dataset. The chatbot's "correct answer" is subjective -- it depends on the counseling stage, the user's emotional state, and whether Bible verses should be included.

**Why It Matters:** Without a `reference` (ground truth), metrics like `ContextRecall` cannot be calculated at all.

### 2.2 Gap: Dual-Source Retrieval (QnA + Bible FAISS)

**The Problem:** Vanilla RAGAS assumes a single retrieval source (e.g., one vector database of documents). This chatbot retrieves from **two independent FAISS indices**:

| Source            | Purpose                                                                                 | Active In                                                                            |
|-------------------|-----------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|
| QnA FAISS Index   | Retrieves a similar counselor response as an "example answer" to inspire the LLM's tone | All stages                                                                           |
| Bible FAISS Index | Retrieves the most relevant Bible verse for spiritual guidance                          | Only <code>solusi</code> , <code>relaksasi</code> , <code>bantuan_profesional</code> |

RAGAS has no built-in concept of "conditional retrieval" that activates only in certain stages.

### 2.3 Gap: Stage-Dependent Behavior Rules

**The Problem:** The chatbot follows a 6-stage counseling protocol managed by `SessionManager`. Each stage has different behavioral rules:

- **pembukaan:** Greet warmly, no advice, no Bible verses.
- **pembahasan:** Listen with empathy, ask open questions, no Bible verses.
- **intervensi:** Reframe negative thinking, no Bible verses.
- **solusi:** Provide practical steps, include Bible verse if relevant.
- **relaksasi:** Calm the user with Bible-based meditation.
- **penutupan:** Summarize and close, no new Bible verses.

A single global RAGAS score cannot tell you whether the chatbot is violating stage rules (e.g., inserting Bible verses during pembukaan). Stage-level analysis is required.

### 2.4 Gap: Default Judge Model (OpenAI GPT-4)

**The Problem:** Vanilla RAGAS defaults to OpenAI's GPT-4 as the LLM judge. This project does not use OpenAI; it uses: - **Google Gemini 2.5-Flash** as the generator (the chatbot's LLM). - **Groq API** (Llama-3) for auxiliary tasks.

Using Gemini as both the generator AND the judge would create **self-evaluation bias** (the model grades its own work favorably). Using GPT-4 requires an API key the project doesn't have.

### 2.5 Gap: Bahasa Indonesia Domain

**The Problem:** RAGAS was designed and tested primarily on English-language tasks. This chatbot operates entirely in **Bahasa Indonesia**, including the training data, the Bible corpus, and the user interactions. The judge LLM must be capable of understanding Indonesian text to produce meaningful evaluations.

## 3. Step-by-Step: Modifying Vanilla RAGAS for This Project

This section walks through how `evaluation/eval_ragas.py` transforms vanilla RAGAS into a project-specific evaluation pipeline.

### Step 1: Install Dependencies

Vanilla RAGAS installs with `pip install ragas`. For this project, additional packages are needed:

```
# Activate the project virtual environment c:\env\Scripts\activate # Install RAGAS (skip scikit-network C++ build on Windows) pip install ragas --no-deps pip install langchain-groq datasets diskcache appdirs instructor rich typer docstring-parser
```

| Package               | Why It's Needed                                 |
|-----------------------|-------------------------------------------------|
| ragas==0.4.3          | Core evaluation framework                       |
| langchain-groq==1.1.2 | LangChain wrapper for Groq API (the judge LLM)  |
| datasets==4.8.5       | HuggingFace datasets library (RAGAS dependency) |

### Step 2: Build the Ground Truth Dataset (Solving Gap 2.1)

Since no pre-built evaluation dataset exists, the script implements a **two-phase workflow**.

#### Phase 1 -- Template Generation:

```
# eval_ragas.py, lines 125-194 # 1. load the QnA counseling dataset df = pd.read_csv("data/dataset_qna.csv") # 2. Randomly sample 50 entries (seed=42 for reproducibility) sample_df = df.sample(n=50, random_state=42) # 3. Auto-assign counseling stages using regex heuristics stages = sample_df["question"].apply(classify_stage) # 4. Save template CSV with EMPTY 'reference' column
```

```
out_df = pd.DataFrame({ "question": sample_df["question"].values, "stage": stages.values, "reference": "", # <-- User fills this manually "intent_override": "", # <-- Optional: for Bible verse stages }) out_df.to_csv("evaluation/data/ragas_testset.csv")
```

The `classify_stage()` function (lines 55-122) uses regex patterns to guess which counseling stage each question belongs to:

```
def classify_stage(question: str) -> str: q = question.strip().lower() # Greetings -> pembukaan if q in ("halo", "hai", "hi"): return "pembukaan" # Closure signals -> penutupan penutupan_patterns = [ r"terima\s*kasih.*kak.*akan\s*(coba|mencoba)", r"terima\s*kasih.*akan\s*coba", # ... more patterns ] # Action/agreement signals -> solusi solusi_patterns = [ r"(saya|aku)\s*(akan\s*)?(coba|mencoba)", r"bisa\s*dicoba", # ... more patterns ] # Everything else -> pembahasan (problem-sharing) return "pembahasan"
```

The output CSV (`ragas_testset.csv`) looks like this:

| question               | stage      | reference                | intent_o                        |
|------------------------|------------|--------------------------|---------------------------------|
| "Halo"                 | pembukaan  | (empty -- fill manually) |                                 |
| "Saya merasa sedih..." | pembahasan | (empty -- fill manually) |                                 |
| "Itu bisa dicoba..."   | solusi     | (empty -- fill manually) | Menyata<br>Perasaan<br>Sedih... |

Step 3: Execute the RAG Pipeline Per Sample (Solving Gap 2.2)

In vanilla RAGAS, you provide `retrieved_contexts` directly. In this project, the script **runs the actual RAG pipeline** on each test question and captures what the engine retrieves:

```
# eval_ragas.py, lines 258-289 # Run the chatbot's RAG engine rag_result = rag_engine.generate_response( user_input=question, current_stage=stage, # Stage-specific behavior override intents=override_intents # For Bible verse retrieval ) # Extract the response and context response = rag_result["response"] ctx = rag_result["context_used"] # Build the retrieved_contexts array from dual sources retrieved_contexts = [] if ctx.get("example_answer"): retrieved_contexts.append(ctx["example_answer"]) # Source 1: QnA FAISS if ctx.get("bible_verses"): retrieved_contexts.append(ctx["bible_verses"]) # Source 2: Bible FAISS # Fallback for stages with no retrieval if not retrieved_contexts: retrieved_contexts = ["(no context retrieved)"]
```

**Key difference from vanilla:** The `retrieved_contexts` list can have 1 or 2 items depending on the stage: - **pembukaan/pembahasan/intervensi/penutupan:** `[example_answer]` (1 context) - **solusi/relaksasi:** `[example_answer, bible_verse]` (2 contexts)

Step 4: Replace the Default Judge with Groq (Solving Gap 2.4)

Vanilla RAGAS looks for `OPENAI_API_KEY` and defaults to GPT-4. The script overrides this entirely:

```
# eval_ragas.py, lines 334-348 from langchain_groq import ChatGroq # Configure Groq Llama-3 as the judge judge_llm = ChatGroq( model="llama-3.1-8b-instant", api_key=os.getenv("GROQ_API_KEY"), temperature=0.0, # Deterministic judging for reproducibility )
```

**Why Groq Llama-3?** 1. **No self-evaluation bias** -- The generator is Gemini 2.5-Flash; the judge is Llama-3. They are independent models. 2. **Higher rate limits** -- Groq's free tier has generous rate limits compared to Gemini Flash. 3. **Already available** -- `GROQ_API_KEY` is already in the project's `.env` file from earlier augmentation work.

Each metric receives the custom judge explicitly:

```
# eval_ragas.py, lines 351-356 metrics = [ Faithfulness(llm=judge_llm), AnswerRelevancy(llm=judge_llm), ContextPrecisionWithoutReference(llm=judge_llm), ContextRecall(llm=judge_llm), ]
```

## Step 5: Add Stage-Level Analytics (Solving Gap 2.3)

After RAGAS computes per-sample scores, the script groups results by counseling stage:

```
# eval_ragas.py, lines 417-431
stage_metrics = []
for stage in sorted(results_df["stage"].unique()):
    stage_mask = results_df["stage"] == stage
    stage_row = {"stage": stage, "sample_count": stage_mask.sum()}
    for col in metric_names:
        values = results_df.loc[stage_mask, col].dropna()
        stage_row[f"{col}_mean"] = values.mean()
        stage_row[f"{col}_std"] = values.std()
    stage_metrics.append(stage_row)
stage_df = pd.DataFrame(stage_metrics)
stage_df.to_csv("evaluation/results/ragas_per_stage.csv")
```

This produces a per-stage CSV that answers questions like: - "Does the chatbot maintain high Faithfulness during the `solusi` stage when Bible verses are injected?" - "Is Context Precision lower during `pembahasan` where only QnA retrieval is active?"

## Step 6: Assemble Everything Into RAGAS Format

Finally, the script maps the collected data into RAGAS's `SingleTurnSample` objects:

```
# eval_ragas.py, lines 321-331
from ragas import evaluate, EvaluationDataset, SingleTurnSample
samples = []
for r in results:
    sample = SingleTurnSample(
        user_input=r["question"], # From test CSV
        response=r["response"], # From RAG pipeline execution
        retrieved_contexts=r["retrieved_contexts"], # From dual FAISS retrieval
        reference=r["reference"], # From manual ground truth
    )
    samples.append(sample)
eval_dataset = EvaluationDataset(samples=samples)
ragas_results = evaluate(dataset=eval_dataset, metrics=metrics)
```

## Summary: Vanilla vs. Modified RAGAS

| Aspect             | Vanilla RAGAS          | This Project's RAGAS                                        |
|--------------------|------------------------|-------------------------------------------------------------|
| Ground Truth       | Pre-existing dataset   | Two-phase: auto-generate template, manually fill references |
| Retrieval Source   | Single vector DB       | Dual FAISS (QnA + Bible), stage-conditional                 |
| Context Collection | Pre-provided           | Auto-collected by running RAGEngine per sample              |
| Judge LLM          | OpenAI GPT-4 (default) | Groq Llama-3.1-8b-instant (explicit override)               |
| Result Granularity | Global scores only     | Global + per-stage + per-sample breakdowns                  |
| Language           | English (default)      | Bahasa Indonesia                                            |
| Stage Awareness    | None                   | 6-stage counseling protocol with stage column               |

## 4. How to Run RAGAS Evaluation in This Project

### Prerequisites

- Virtual environment activated:** `cbenv\Scripts\activate`
  - Dependencies installed:** `ragas`, `langchain-groq` (see Step 1 in Section 3)
- API keys in `.env`:**
- `GEMINI_API_KEY` -- for the chatbot's RAG pipeline (Gemini 2.5-Flash)
  - `GROQ_API_KEY` -- for the RAGAS judge LLM (Llama-3)

- 4. **FAISS indices built:** The Bible and QnA indices must exist (auto-built on first RAGEngine startup)
- 5. **Model checkpoint:** checkpoint/IndoBERT\_multi\_label\_zsl.pt must be present

Phase 1: Generate the Test Template

```
python evaluation/eval_ragas.py --generate
```

**What happens:** 1. Reads data/dataset\_qna.csv (367 QnA pairs) 2. Randomly samples 50 entries (seed=42) 3. Auto-assigns counseling stages via regex heuristics 4. Saves evaluation/data/ragas\_testset.csv

**Expected output:**

```
===== RAGAS Test Set Generator (Phase 1) =====
===== QnA dataset loaded: 367 rows Sampled: 50 entries Auto-assigned
stage distribution: pembahasan: 40 solusi: 4 (Bible verse) pembukaan: 3 penutupan: 2 relaksasi: 1 (Bible verse) [OK] Saved:
../evaluation/data/ragas_testset.csv =====
```

Manual Step: Fill In the Reference Column

Open evaluation/data/ragas\_testset.csv in Excel or a text editor and:

- Fill the reference column** with the ideal counselor response for each question:
  - For solusi/relaksasi stages: include the relevant Bible verse in the reference.
  - For other stages: write the ideal empathetic response.
- 2. **Verify the stage column** -- adjust if the auto-assignment is incorrect.
- 3. **Adjust intent\_override** for Bible verse stages if the default intent is not appropriate.

Phase 2: Run the Evaluation

```
python evaluation/eval_ragas.py --evaluate
```

**What happens:** 1. Validates that all reference cells are non-empty (exits with error if any are blank) 2. Initializes RAGEngine (loads FAISS indices, classifier) 3. Runs the RAG pipeline on each of the 50 samples 4. Builds a RAGAS EvaluationDataset from the collected data 5. Configures the Groq judge and runs evaluate() 6. Saves three output CSVs

**Estimated duration:** 10-20 minutes (depends on Gemini API latency and Groq rate limits)

5. How to Read the Results

5.1 Output Files

Phase 2 produces three CSV files in evaluation/results/:

| File                 | Content                                            | Rows                               |
|----------------------|----------------------------------------------------|------------------------------------|
| ragas_per_sample.csv | Per-sample metric scores for all 50 test cases     | 50                                 |
| ragas_summary.csv    | Aggregate statistics (mean, median, std, min, max) | 4 (one per metric)                 |
| ragas_per_stage.csv  | Per-stage average scores                           | Up to 6 (one per counseling stage) |

5.2 Reading ragas\_summary.csv

This file provides the overall performance of the RAG pipeline:

| metric            | mean | median | std  | min  | max  | count |
|-------------------|------|--------|------|------|------|-------|
| faithfulness      | 0.85 | 0.90   | 0.12 | 0.40 | 1.00 | 50    |
| answer_relevancy  | 0.78 | 0.80   | 0.15 | 0.30 | 0.95 | 50    |
| context_precision | 0.72 | 0.75   | 0.18 | 0.20 | 1.00 | 50    |
| context_recall    | 0.65 | 0.70   | 0.20 | 0.10 | 0.95 | 50    |

(Note: The above numbers are illustrative examples, not actual results.)

How to interpret each metric:

Faithfulness (Generator Quality)

- **Score range:** 0.0 (complete hallucination) to 1.0 (fully grounded)
- **What high means:** Every claim in the chatbot's response can be traced back to the retrieved context (QnA answer or Bible verse).
- **What low means:** The chatbot is generating claims or information not present in the retrieved context (hallucination).
- **Thesis interpretation:** "The RAG pipeline achieves X.XX Faithfulness, indicating that the Gemini-generated responses are [well-grounded / partially hallucinating / mostly hallucinating] in the retrieved knowledge base."

Answer Relevancy (Generator Quality)

- **Score range:** 0.0 (completely off-topic) to 1.0 (perfectly relevant)
- **What high means:** The chatbot's response directly addresses the user's expressed concern.
- **What low means:** The response is generic, tangential, or addresses a different problem.
- **Thesis interpretation:** "The chatbot maintains X.XX Answer Relevancy, demonstrating that [the prompt template effectively guides Gemini / the response generation needs improvement]."

Context Precision (Retriever Quality)

- **Score range:** 0.0 (irrelevant documents retrieved) to 1.0 (all retrieved documents are relevant)
- **What high means:** The FAISS indices are returning documents (QnA answers, Bible verses) that are relevant to the user's input.
- **What low means:** The retriever is pulling in irrelevant context, which wastes the LLM's attention.
- **Thesis interpretation:** "The FAISS-based retrieval achieves X.XX Context Precision, confirming that [the paraphrase-multilingual-MiniLM embedding model is appropriate / the embedding model struggles with domain-specific queries]."

Context Recall (Retriever Quality)

- **Score range:** 0.0 (retrieved context covers none of the answer) to 1.0 (retrieved context covers everything needed)
- **What high means:** The retrieved context contains enough information to produce the reference answer.
- **What low means:** Important information is missing from the retrieved context, forcing the LLM to compensate (or hallucinate).
- **Thesis interpretation:** "Context Recall of X.XX indicates that [the knowledge base is comprehensive / there are gaps in the QnA/Bible datasets that need expansion]."

5.3 Reading ragas\_per\_stage.csv

This file reveals stage-specific behavior:

| stage      | sample_count | faithfulness_mean | faithfulness_std | answer_relevancy_mea | ... |
|------------|--------------|-------------------|------------------|----------------------|-----|
| pembahasan | 40           | 0.87              | 0.10             | 0.80                 | ... |
| pembukaan  | 3            | 0.92              | 0.05             | 0.85                 | ... |
| penutupan  | 2            | 0.80              | 0.14             | 0.75                 | ... |
| relaksasi  | 1            | 0.90              | NaN              | 0.70                 | ... |
| solusi     | 4            | 0.82              | 0.11             | 0.78                 | ... |

(Note: The above numbers are illustrative examples.)

### Key comparisons to make:

**solusi/relaksasi vs. other stages:** These stages use Bible verse retrieval. If Context Precision is significantly higher or lower here, it reveals whether the Bible FAISS index is performing well.

**pembukaan Faithfulness:** Should be very high (close to 1.0) because the chatbot should only greet the user, not generate any substantive claims.

**pembahasan Answer Relevancy:** Should be high because the chatbot should be responding empathetically to the user's specific problem.

**Standard deviation:** High std within a stage suggests inconsistent performance -- some queries work well while others fail.

### 5.4 Reading ragas\_per\_sample.csv

This file contains the raw scores for **every individual test case**. Use it for:

- Identifying failure cases:** Sort by `faithfulness` ascending to find samples where the chatbot hallucinated most.
- Diagnosing retrieval failures:** Sort by `context_precision` ascending to find cases where FAISS retrieved irrelevant documents.
- Cross-referencing with stage:** Filter by stage to analyze patterns within specific counseling phases.

### 5.5 Interpreting Results for the Thesis

The overall narrative for the thesis should connect the metrics to the research questions:

| Research Question                                            | Relevant Metric(s)                | What to Report                                                   |
|--------------------------------------------------------------|-----------------------------------|------------------------------------------------------------------|
| Does the chatbot ground its responses in the knowledge base? | Faithfulness                      | Overall mean + std                                               |
| Does the chatbot address the user's specific concern?        | Answer Relevancy                  | Overall mean + per-stage comparison                              |
| Is the FAISS retrieval model (MiniLM) effective?             | Context Precision, Context Recall | Overall mean + comparison between Bible and non-Bible stages     |
| Does the chatbot follow stage-specific behavioral rules?     | All metrics, per-stage            | Compare stage-level scores to validate SessionManager compliance |
| Is the RAG pipeline better than a non-RAG approach?          | Faithfulness, Answer Relevancy    | Qualitative comparison with non-RAG baseline (if available)      |

## 5.6 Score Benchmarks

While RAGAS does not define official "passing" thresholds, general guidelines from the community:

| Score Range | Interpretation                                     |
|-------------|----------------------------------------------------|
| 0.85 - 1.00 | Excellent -- production-ready quality              |
| 0.70 - 0.84 | Good -- acceptable for most applications           |
| 0.50 - 0.69 | Fair -- needs improvement in specific areas        |
| Below 0.50  | Poor -- significant issues requiring investigation |